On-Chip Serial Interface with Low Pin Count

Patrick Kurth

June 2023

1 Overview

This document describes the usage and the implementation of an on-chip serial interface with a low pin count. The intended use of this interface is in circuits requiring some calibration/configuration while having high constraint on the available pads. The original target was high-frequency circuits without packaging (wafer probing of bare dies) where only a handful of pads are available for use. Therefore, the most important parameter of the data interface is low pin count.

The interface presented in this document is functionally implemented in verilog. For the physical implementation, the project uses *openPCells* to generate the required layout views. The entire project is implemented with technology-independent standard cells. Timing issues across technology nodes are solved by using a practically-hold-violation-free architecture. This is achieved by alternating flip-flop clock polarities (a positively edge-triggered flip-flop only follows a negatively edge-triggered and vice versa). This roughly equalized the timing margins for both setup- and hold-violations. Finally, by using a low-frequency clock these margins can be high enough for any technology node.

2 Introduction

3 Interfacing Signals

The top-level schematic of the serial interface is shown in figure 1. The interface has only two off-chip pins (clock and data). VDD and VSS are of course also present but usually shared with other circuits and therefore not included in the pin count. On-chip the data is provided parallel, therefore N pins connect to the serial interface.

The clock is always provided by the driver circuit (off-chip), the data are bidirectional.

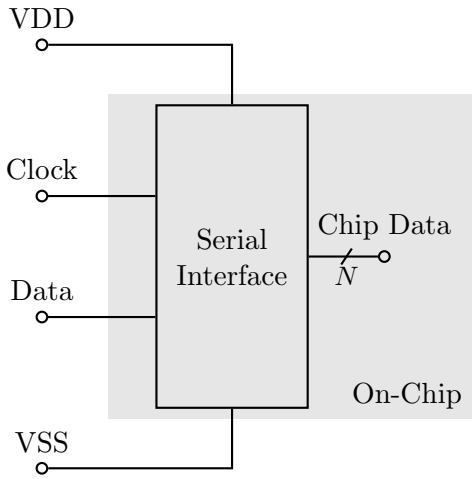


Figure 1: Block Diagram of the Serial Interface

4 Communication Protocol

The serial interface is state-based and command-driven. The following commands are supported and their bit patterns are documented in table 1:

- `RESET`

  Reset the entire register to its default value. This only resets the main registers that face the circuits on the chip. The communication registers are not affected, which means that an update of the main registers following a reset does not necessarily ensure default data.

- `START_SEND`

  Start the transmission of data from the driver to the chip. The data directly follow the command. The interface assumes that the correct number of bits is sent. It is not possible to send any number of bits different from the actual data length of the interface.

- `START_RECEIVE`

  Start the reception of data from the chip to the driver. This is the only state where the chip drives the data line. Following this command, the driver must disable the output buffers that drive the data line, that is, enter a high-Z mode. This must happen in half of a clock cycle (see detailed timing diagram below). The interface then sends out the current N bits that are stored in the interface. Only the data in the communication registers is sent, not what is actually stored in the main registers facing the circuits on the chip. It is not possible to read the data stored in the main registers.

- `UPDATE`

  Apply the data stored in the communication registers to the main registers. The data out of the interface (the bus that goes into the chip) only reflects this data. Without an update, this data is never changed. This ensures that there is no garbage data while the communication registers are being written. Usually an update directly follows a send command.

Command	Pattern
`RESET`	`00`
`START_SEND`	`01`
`START_RECEIVE`	`01`
`UPDATE`	`01`

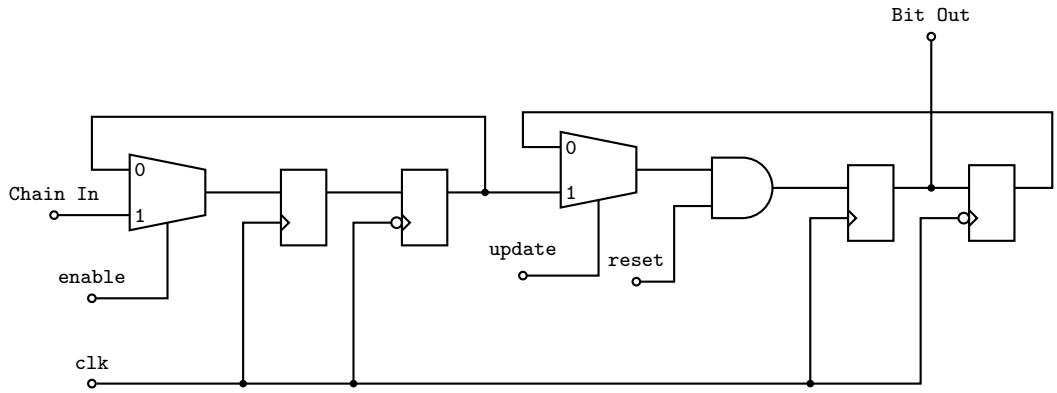
Table 1: Bit Patterns of Commands

All commands are preceeded by a start sequence. The default (and currently implemented) sequence is `101`.

5 Implementation

5.1 Register Cells

5.1.1 Register Cell 0



5.1.2 Register Cell 1